INFORMATIK · KLASSE 9

LÖSUNGEN

# Aufgabe 3 – Datensätze filtern mit WHERE

Sicherungsblatt

## 1 · Von users zur gefilterten Ergebnisrelation

Ausschnitt aus users (6 von 317 Zeilen; 3 von 18 Spalten)

| id | username | city |
|---|---|---|
| 1 | zornigerfotograf | Schwerin |
| 2 | wanderwut_finn | München |
| 3 | bergcoder | Zürich |
| 4 | koch_kicker | Berlin |
| 8 | theaterwut_oscar | Leipzig |
| 34 | gartenlinse | Berlin |

Ergebnisrelation dieses Ausschnitts (2 von 6 Zeilen)

| username | city |
|---|---|
| koch_kicker | Berlin |
| gartenlinse | Berlin |

**Arbeitsauftrag:** Zeige die Benutzernamen und Wohnorte der Mitglieder aus Berlin.

SQL-Anweisung

```
SELECT username, city
FROM users
WHERE city = 'Berlin'
```

Allgemeine Form

```
SELECT Spaltenname
FROM tabellenname
WHERE Bedingung
```

Dies sind die Berliner Treffer im gezeigten Ausschnitt, nicht alle Berliner Mitglieder der Datenbank.

**Merke**

WHERE wählt die Zeilen aus, die eine Bedingung erfüllen. Diese Auswahl heißt Selektion. Die gespeicherte Tabelle bleibt unverändert.

## 2 · Einfache Bedingungen: Texte und Zahlen

**Arbeitsauftrag:** Liste die Daten aller weiblichen Mitglieder auf.

SQL-Anweisung

```
SELECT *
FROM users
WHERE gender = 'female'
```

Ausschnitt: 1 von 148 Zeilen, 3 von 18 Spalten

| id | name | gender |
|---|---|---|
| 151 | Mia Sonntag | female |
| ... | ... | ... |

**Arbeitsauftrag:** Gib alle Mitglieder aus, die kleiner als 1,80 m sind.

SQL-Anweisung

```
SELECT *
FROM users
WHERE centimeters < 180
```

Ausschnitt: 1 von 245 Zeilen, 3 von 18 Spalten

| id | name | centimeters |
|---|---|---|
| 1 | Noah Schneider | 168 |
| ... | ... | ... |

**Merke**

Textwerte stehen in einfachen Anführungszeichen, Zahlen nicht. < 180 schließt genau 180 cm aus, <= 180 schließt sie ein.

## 3 · Vergleiche und Datumswerte

Vergleichsoperatoren

| = gleich | < kleiner als | > größer als | <= kleiner oder gleich | >= größer oder gleich | != / <> ungleich |
|---|---|---|---|---|---|

**Arbeitsauftrag:** Finde das Mitglied, das am 17.09.2007 geboren wurde. Gib den vollständigen Datensatz aus.

SQL-Anweisung

```
SELECT *
FROM users
WHERE birthday = '2007-09-17'
```

Ergebnisrelation: 1 Zeile; 3 von 18 Spalten gezeigt

| id | name | birthday |
|---|---|---|
| 5 | Ben Neumann | 2007-09-17 |

**Arbeitsauftrag:** Gib die Namen aller Mitglieder aus, die nicht in Leipzig wohnen.

SQL-Anweisung

```
SELECT name
FROM users
WHERE city != 'Leipzig'
```

Ausschnitt: 1 von 305 Zeilen

| name |
|---|
| Noah Schneider |
| ... |

Datumswerte stehen im Format 'JJJJ-MM-TT'. NULL bedeutet: kein Wert eingetragen. Ein normaler Vergleich mit NULL erfüllt die WHERE-Bedingung nicht. Deshalb fehlt bei city != 'Leipzig' auch das Mitglied ohne eingetragenen Wohnort.

LÖSUNGEN

# Aufgabe 3 – Datensätze filtern mit WHERE

Sicherungsblatt

## 4 · Muster suchen mit LIKE

**Arbeitsauftrag:** Zeige alle Mitglieder, deren Vorname mit B beginnt.

SQL-Anweisung

```
SELECT *
FROM users
WHERE name LIKE 'B%'
```

Ergebnisrelation: 1 Zeile; 3 von 18 Spalten gezeigt

| id | name | username |
|---|---|---|
| 5 | Ben Neumann | modegamer_ben |

**Arbeitsauftrag:** Zeige alle Mitglieder, deren Benutzername die Zeichenfolge „fuchs“ enthält.

SQL-Anweisung

```
SELECT *
FROM users
WHERE username LIKE '%fuchs%'
```

Ausschnitt: 1 von 7 Zeilen; 3 von 18 Spalten

| id | name | username |
|---|---|---|
| 33 | Yannick Vogt | reisefuchs_yannick |
| ... | ... | ... |

**Arbeitsauftrag:** Gib die Namen aller Mitglieder aus, die im März Geburtstag haben.

SQL-Anweisung

```
SELECT name
FROM users
WHERE birthday LIKE '%-03-%'
```

Ausschnitt: 1 von 34 Zeilen

| name |
|---|
| Timo Reichert |
| ... |

Muster im Vergleich

| | |
|---|---|
| `name LIKE 'B%'` | beginnt mit B |
| `name LIKE '%berg'` | endet mit berg |
| `username LIKE '%fuchs%'` | enthält fuchs |

**Merke**

LIKE vergleicht mit einem Muster. % steht für beliebig viele Zeichen, auch für kein Zeichen. Die Position von % bestimmt, wo weitere Zeichen stehen dürfen.

name enthält Vor- und Nachnamen, username den Benutzernamen. '%-03-%' sucht März an der Monatsposition; '%03%' wäre zu ungenau.

## 5 · Bedingungen verknüpfen mit AND und OR

**Arbeitsauftrag:** Finde alle Mitglieder aus Berlin, deren Vorname Lina lautet.

SQL-Anweisung

```
SELECT *
FROM users
WHERE name LIKE 'Lina %'
  AND city = 'Berlin'
```

Ergebnisrelation: 1 Zeile; 3 von 18 Spalten gezeigt

| id | name | city |
|---|---|---|
| 196 | Lina Gerstenberg | Berlin |

**Arbeitsauftrag:** Liste alle Mitglieder auf, deren Vorname Lina oder Felix lautet.

SQL-Anweisung

```
SELECT *
FROM users
WHERE name LIKE 'Lina %'
  OR name LIKE 'Felix %'
```

Ausschnitt: 2 von 8 Zeilen; 3 von 18 Spalten

| id | name | city |
|---|---|---|
| 6 | Felix Zimmermann | Hannover |
| 153 | Lina Eichinger | Saarbrücken |
| ... | ... | ... |

**Merke**

AND verlangt, dass beide Bedingungen erfüllt sind. Bei OR genügt mindestens eine Bedingung; auch beide dürfen erfüllt sein. Jede Teilbedingung muss vollständig sein.

Das Leerzeichen vor % in 'Lina %' und 'Felix %' begrenzt die Suche auf den vollständigen Vornamen.

## 6 · Klammern und Klauselreihenfolge

**Arbeitsauftrag:** Zeige alle Mitglieder aus Berlin, deren Vorname Lina oder Hailey lautet.

SQL-Anweisung

```
SELECT *
FROM users
WHERE (name LIKE 'Lina %'
  OR name LIKE 'Hailey %')
  AND city = 'Berlin'
```

Ergebnisrelation: 2 Zeilen; 3 von 18 Spalten gezeigt

| id | name | city |
|---|---|---|
| 196 | Lina Gerstenberg | Berlin |
| 200 | Hailey Kronenberg | Berlin |

Bekannte Klauselreihenfolge

**SELECT → FROM → WHERE → ORDER BY → LIMIT**

SQL-Anweisung

```
SELECT username, centimeters
FROM users
WHERE centimeters < 180
ORDER BY centimeters DESC
LIMIT 5
```

Zuerst werden passende Zeilen ausgewählt, danach werden sie für die Ausgabe sortiert und begrenzt.

**Merke**

AND bindet stärker als OR. Die Klammern sorgen hier dafür, dass die Stadtbedingung für beide Vornamen gilt.